

Sarvam 30B / 105B: architecture study and nanosarvam experiment log

A line-by-line reading of Sarvam AI's released modelling code, and what a 1/1000th-scale re-implementation measured about each design choice.

Generated 09 September 2026 from the project's Markdown sources. Line references are into `sarvam/sarvam-30b/` and `sarvam/sarvam-105b/`; checkpoint names refer to `nanosarvam/`.

Contents

1. Part I — How Sarvam 30B and 105B are built (docs/ARCHITECTURE.md)

- 1. Provenance
- 2. Shape
- 3. Attention: two different answers
- 4. Long context: opposite strategies
- 5. MoE routing
- 6. The released 105B file cannot train
- 7. Other things that will bite you
- 8. Reported benchmarks
- 9. The wider stack
- 10. Rebuilt at 1/1000th scale: what held up
- Sources

2. Part II — Experiment log (docs/RESULTS.md)

- The two presets
 - 1. English (tinyshakespeare, 1.12 MB)
 - 2. Hindi
 - 3. Does YaRN earn its place?

- 4. MoE dispatch: `sparse` vs `dense`
- 5. Load balancing behaviour
- 6. Scaling to the GPU: sizing `nano_350m`
- 7. The tokenizer experiment: bytes vs BPE at matched compute
- 8. Corpus cleaning: v2
- Reproducing
- 9. The v4 recipe, A/B at matched compute, on a shared benchmark
- 10. Architecture ablations on the v4 recipe
- 11. The data lever: the whole dump at the same compute
- 12. The ablation winners together
- 13. Prose filtering: a clean negative
- 14. Out of register: Hindi Wikisource
- 15. What the sliding window costs past the training length
- 16. `bias_update_rate` for fine-grained experts
- 17. Closing the register gap: Wikisource as training data
- 18. More steps on more data: the mixed corpus at 3× the budget
- 19. Accuracy: a golden set
- 20. Fine-tuning to answer questions
- 21. Twice the steps: 18k → 36k, and what reaches the answers

How Sarvam 30B and 105B are built

Source: docs/ARCHITECTURE.md

Findings from reading Sarvam AI's released modelling code and configs. Every number here comes from sarvam/sarvam-*/config.json or the modelling files, not from marketing material. Where a claim comes from a blog post or press coverage instead, it says so.

Line references are into the downloaded originals under sarvam/. The annotated companions in that directory walk through the same code with tensor shapes on every line.

Sections 1–9 are the reading. Section 10 is what happened when the same architecture was rebuilt at 1/1000th scale in nanosarvam/ and trained — which of these mechanisms held up, which did not, and how much any of them matter next to the tokenizer and the data.

1. Provenance

Released	February 2026, Apache 2.0
Trained	Entirely in India on IndiaAI Mission compute — 4,096 NVIDIA H100 SXM via Yotta
Pretraining	16T tokens (30B), ~12T (105B)
Post-training	In-house SFT, then asynchronous GRPO with KL regularization deliberately dropped
Tokenizer	262,144 tokens, 22 scheduled Indian languages across 12 scripts
Lineage	Sarvam-1 (2B, Oct 2024, from scratch) → Sarvam-M (24B, May 2025, a Mistral-Small post-train) → these two (from scratch)

The models are **DeepSeek-lineage**. The 105B file says so in its header: *"based on Llama and Deepseek MoE implementations... modified to accommodate Sarvam's MLA MoE architecture."* MLA, fine-grained MoE with a shared expert, the sigmoid noaux_tc router — all DeepSeek-V2/V3 mechanisms. The original contributions are the tokenizer, the Indic data pipeline, and doing the whole run domestically.

2. Shape

Both are sparse Mixture-of-Experts decoders. Shared: hidden_size 4096, 128 experts + 1 shared expert, layer 0 dense (first_k_dense_replace: 1), QK-norm, sigmoid router with aux-loss-free balancing, routed_scaling_factor 2.5, vocab 262,144, untied embeddings.

	Sarvam 30B	Sarvam 105B
class	SarvamMoEForCausalLM	SarvamMLAForCausalLM
total params	~32 B	~106 B
active per token	2.4 B non-embedding	10.3 B

	Sarvam 30B	Sarvam 105B
layers	19	32
attention	GQA , 64 q-heads / 4 kv-heads, head_dim 64	MLA , 64 heads
MLA dims	—	kv_lora_rank 512, q_head_dim 192 (128 noPE + 64 RoPE), v_head_dim 128
experts per token	top-6 of 128	top-8 of 128
moe_intermediate_size	1024 (hidden/4)	2048 (hidden/2)
dense layer-0 FFN	8192	16384
long context	rope_theta 8e6, no scaling, 65K eval	rope_theta 1e4 + YaRN factor 40, 128K
group-limited routing	disabled (n_group=1)	16 groups, keep 2 — see §5

Parameter arithmetic checks out against the published figures. Counting SwiGLU experts as $3 \cdot D \cdot I$ over 18/31 MoE layers plus one dense layer, attention, and untied embeddings: **106.0 B / 10.25 B active** and **32.1 B / 2.36 B**. So "30B" and "105B" are round-number product names; the configs give 32B and 106B.

Per token only **9 of 129** expert MLPs fire in the 105B (8 routed + 1 shared), **7 of 129** in the 30B. That ratio is the whole economic argument.

3. Attention: two different answers

MLA (105B) — sarvam-105b/modeling_sarvam_moe.py:467

One projection, kv_a_proj_with_mqa, emits kv_lora_rank + qk_rope_head_dim = 512 + 64 = **576 values per token, total, for all 64 heads**. That is the entire KV cache. kv_b_proj expands 512 → $64 \times (128 + 128)$ at compute time.

Two details that matter:

- **The positional key has head dimension 1.** It is MQA-style, one copy shared by every head — that's the _with_mqa in the projection's name. It is also why RoPE can be applied *before* the latent is expanded: the rotation touches only the 64-d shared slice, so the 512-d latent stays position-free and cacheable.
- **Query and key are concatenated, not summed:** $q = [q_{\text{nope}} \parallel q_{\text{pe}}]$. So $q \cdot k$ decomposes into $\langle q_{\text{nope}}, k_{\text{nope}} \rangle + \langle q_{\text{pe}}, k_{\text{pe}} \rangle$ — a position-free content term plus a purely relative positional term. Keys are 192-d, values 128-d; that asymmetry breaks any code assuming one head_dim.

Caveat that will mislead you if you benchmark the released file: it caches the *expanded* keys and values (line 610) — $64 \times (192+128) = 20,480$ values per token per layer. Real MLA caches 576, which is 35.6× smaller and is where config.json's otherwise-unused head_dim: 576 comes from. vLLM and SGLang get there by absorbing kv_b_proj into q_proj and o_proj so kv is never materialised. Same algebra, reassocated. **The HF file shows the math, not MLA's memory profile.**

GQA (30B) — sarvam-30b/modeling_sarvam_moe.py:371

Conventional by comparison. Fused QKV of width $(64 + 2 \cdot 4) \times 64 = 4608$, split on the head axis, QK-norm (RMSNorm over the 64-d head, before RoPE), then repeat_kv broadcasts 4 KV heads to 64. Unlike the 105B

file, the shipped code genuinely realises its saving: it caches 4 heads and expands afterward.

KV cache per token per layer:

	values
30B GQA	512
105B MLA (absorbed, as served)	576
105B MLA (as the HF file runs)	20,480

Across full depth: 9,728/token for the 30B against 18,432 for the 105B — under 2× the cache for 3.3× the parameters. That is what MLA buys, and why the 105B gets the 128K window.

4. Long context: opposite strategies

RoPE's frequency ladder is two mechanisms, not one. Fast dims wrap many times inside the training window, so they encode *local offset*; squash them and neighbouring tokens blur. Slow dims don't complete one rotation, so they encode *absolute position*; extend past training and you request phases never seen.

- **30B: pure extrapolation.** `rope_theta = 8e6`, `rope_scaling: null`. Spread the ladder wide enough during training that long positions are in distribution. Works only because they trained that way.
- **105B: YaRN (NTK-by-parts).** `rope_theta = 1e4`, factor 40, from a 4096 base window to 131072. Keep the fast dims, divide the slow dims by 40, ramp linearly between.

Worked out for the 105B's config (`qk_rope_head_dim 64` → 32 frequency pairs):

```
correction_dim(beta_fast=32) = 10.4722 → low = 10
correction_dim(beta_slow=1) = 22.5134 → high = 23
```

11 of 32 dims untouched, 13 blended, 8 fully interpolated. Dim 10 turns 36.7× over the 4096 window (just past `beta_fast=32`); dim 23 turns 0.9× (just under `beta_slow=1`). The cutoffs land exactly where asked.

Three traps here:

1. **The temperature term is in a different class.** `_mscale` in the rotary is `get(40, mscale) / get(40, mscale_all_dim)` = **exactly 1.0**, so the cos/sin tables are unscaled. The correction lives in `SarvamMLAAttention.__init__:522` instead: `softmax_scale = 192^-0.5 × 1.36889² = 0.13523`, a 1.874× gain compensating for attention entropy at 40× context. Read only the rotary file and you would conclude the term was unused.
2. **`beta_fast / beta_slow` are measured against `original_max_position_embeddings (4096)`, never the **131072 target**.** Most-often-mis-set field when copying a `rope_scaling` block between models.
3. **`factor=40` for a 32× extension** (131072/4096). Deliberate margin; nothing cross-checks them.

5. MoE routing

`MoEGate.forward` (105B :293) is DeepSeek-V3's `noaux_tc`, and the **sigmoid is load-bearing**. Because experts are scored independently rather than competing in a softmax, you can add a per-expert

`e_score_correction_bias` — nudged down for overloaded experts, up for starved ones — without distorting the others.

Crucially the bias steers **selection only**: `topk_weight` is gathered from the *unbiased* scores. Balancing never enters the loss. No auxiliary load-balancing term fighting the language-modelling objective.

Dispatch is one line: `idxs = topk_ids.view(-1).argsort()`, then `x[idxs // k]`. Sorting assignments by expert makes each expert's work contiguous — 128 dense GEMMs instead of per-token gather — then `new_x[idxs] = outs` unsorts.

The `n_group` trap

Same gate code, opposite behaviour:

- **30B**: `n_group=1`, `topk_group=1`, explicit in both `config.json` and the config class. Grouping is a no-op — plain top-6 of 128.
- **105B**: `n_group` and `topk_group` appear **nowhere** — not in `config.json`, not in `SarvamMLAConfig.__init__`. The gate reads them via `getattr(config, "n_group", num_experts // 8)` → **16** and `getattr(config, "topk_group", 2)` → **2**.

So the shipped 105B routes into 16 groups of 8, keeps the best 2 groups (16 candidate experts), then takes top-8 of those. **All 8 experts for a token must come from 2 of 16 groups** — a hard constraint on the router that exists only as a `getattr` default. Add an `n_group` field to that config for any other purpose and you silently change the model's routing.

Group-limited routing exists for device placement: with experts sharded across nodes, unconstrained top-k could touch every node for one token. Capping groups caps the all-to-all fan-out.

6. The released 105B file cannot train

Four independent blocks, worth knowing before anyone plans a fine-tune:

1. `MoEGate.forward:305` — `assert not self.training`
2. `moe_infer` is decorated `@torch.no_grad()`
3. `SarvamMLAMoE.forward`'s if/else calls `moe_infer` on **both** branches, under a comment conceding *"in practice, you'd want a more sophisticated training implementation"*
4. `supports_gradient_checkpointing = True` and `self.gradient_checkpointing` is initialised, but the model loop **never calls** `_gradient_checkpointing_func`. `gradient_checkpointing_enable()` reports success and does nothing.

The 30B file has a working training path (boolean-mask gather over `repeat_interleave`) and real gradient checkpointing.

7. Other things that will bite you

- **The 105B is eager-attention only.** `_supports_flash_attn_2 = False` # Not implemented yet, no SDPA subclass, no dispatch table, and `_prepare_4d_causal_attention_mask` unconditionally. It

materialises (B, 64, S, T) attention weights — at 128K that is ~2.2 PB per batch element. The 128K window is real but belongs to vLLM/SGLang, not this file.

- **.generate() asymmetry.** The 30B declares `GenerationMixin`; the 105B does not. Since Transformers v4.50 `PreTrainedModel` no longer provides it, and `config.json` pins 4.57.2 — consistent with the 105B still carrying hand-written `prepare_inputs_for_generation` and `_reorder_cache`. *Not verified empirically here* (no `transformers` install); check on yours.
- **`_tied_weights_keys = ["lm_head.weight"]` is inert in both** — both configs set `tie_word_embeddings: false`, and HF only acts on the attribute when the flag is true. Head and embedding are two separate 262144×4096 matrices, 2.15 GB, 6.7% of the 30B's parameters.
- **Per-layer RoPE in the 105B.** Each attention layer owns a rotary module and builds a table at `max_position_embeddings` on construction: 67.1 MB per layer × 32 = **2.15 GB allocated and immediately orphaned** at load.
- **The 105B permutes rope dims before rotating** (`view(...,32,2).transpose(4,3)`), the 30B does not. The 105B checkpoint stores rope dims interleaved (GPT-J convention) while `rotate_half` expects split-halves (GPT-NeoX). Port the weights without that permutation and the model looks fine on short prompts and decays with distance.
- **`max_window_layers: 19` in the 30B config is a dead field.** It is Qwen2's knob for "sliding window on the first K layers", and the 30B ships it set to its own layer count — but with no `sliding_window` alongside it, and `modeling_sarvam_moe.py` never reads either. Every layer attends to the full context. `nanosarvam` implements the semantics the field implies (`--sliding-window W --max-window-layers K`) and \$10 has what it costs: nothing.
- **Naming is the practical hazard.** Same company, same release, same filename: `embed_tokens / self_attn / SarvamMLA*` versus `word_embeddings / attention / SarvamMoE*`. Nothing that walks one state dict walks the other. Treat them as two codebases.

8. Reported benchmarks

From the model cards — not reproduced here.

	MMLU	Math500	HumanEval	BrowseComp	τ^2 -bench	SWE-Bench
Sarvam 105B	90.6	98.6	—	49.5	68.3	45.0
Sarvam 30B	85.1	97.0	92.1	35.5	—	—

9. The wider stack

The LLMs are one layer. The Indic-specific work is arguably the moat: **Saaras v3** (ASR, 23 languages, with transcribe/translate/verbatim/translit/codemix modes), **Bulbul v3** (TTS tuned for Indian prosody), **Mayura** and **Sarvam-Translate**, **Sarvam Vision** (OCR, 23 languages), **Sarvam Edge** (on-device). Products: **Samvaad** (voice agents, on the 30B) and **Indus** (agent suite, on the 105B).

The tokenizer deserves emphasis. Sarvam-1's card reports fertility of 1.4–2.1 tokens per word across Indic languages, 2–4× better than Llama/Gemma tokenizers. Every saved token is attention cost, KV cache, and latency. `docs/RESULTS.md` measures both sides of that. The *absence* of such a tokenizer (\$2): byte-level Hindi

runs at 2.48 bytes per character, so a 512-byte window holds only ~206 Devanagari characters. And the *presence* of even a crude one (§7): a 16k-token byte-level BPE trained in ten seconds on 12 MB of Hindi reaches 1.34 tokens per word — inside Sarvam's reported range — and at identical compute is worth 0.162 bits/byte, more than a 20× parameter increase bought (0.110). It is the largest single lever in the whole experiment log, ahead of anything in §2–§5.

10. Rebuilt at 1/1000th scale: what held up

`nanosarvam/` re-implements §2–§5 as trainable pure PyTorch (no HuggingFace) and trains it, mostly on Hindi Wikipedia and Wikisource. The presets keep each real model's *ratios* — `moe_intermediate_size` as a fraction of hidden, the dense-layer FFN, the KV-to-query ratio, `n_group`, `rope_theta` — not its size. `nano_350m` is the GPU workhorse (sized to an 8 GB card, RESULTS §6); the "combo" column is the configuration the ablations converged on, run as `--cfg` overrides of that preset, and is the model to build on.

	30B → <code>nano_30b</code>	105B → <code>nano_105b</code>	<code>nano_350m</code>	combo (RESULTS §12)
total / active params	32.1B / 2.36B → 17.4M / 6.6M	106B / 10.3B → 47.4M / 13.3M	386M / 185M +	364M / 140M
layers	19 → 8	32 → 12	20	20
attention	GQA 64q/4kv → 6q/2kv, head_dim 64	MLA, latent 512 + rope 64 → 48 + 24	GQA 16q/4kv, d 64	same
experts	128 top-6 +1 shared → 16 top-2 +1	128 top-8 +1 → 16 top-2 +1	12×384 top-3 +1	24×192 top-4, no shared
<code>moe_intermediate_size</code>	hidden/4 → hidden/4	hidden/2 → hidden/2	3·hidden/8	hidden-3/16
dense layer-0 FFN	2·hidden → 2·hidden	4·hidden → 4·hidden	2816	2816
group-limited routing	off → off	16 keep 2 (getattr) → 4 keep 2	off	off
RoPE	θ 8e6, no scaling → θ 1e6	θ 1e4 + YaRN 40 → θ 1e4, YaRN on demand	θ 1e6	θ 1e6
sliding window	dead field (§7) → off	—	off	256 on layers 0–9
tokenizer	262k, 22 languages → bytes or 16k BPE	same	16k BPE	16k BPE

+ with the 16k BPE head; 353M / 152M with the byte vocabulary. Sparsity is milder than the originals throughout — 16 experts top-2 is 12.5% where the real models route ~5% — because top-1 of 16 would be degenerate.

Faithful: the sigmoid router, the bias-only balancer with the bias kept out of the combine weights, group-limited top-k, the shared expert and dense layer 0, MLA's single `kv_a_proj_with_mqa` with RoPE applied before expansion, QK-norm before RoPE, untied embeddings, fp32 router logits, and YaRN with the temperature term on the attention scale. Deliberately different: it trains, its KV cache actually caches the 576-style latent rather than the expanded heads, RoPE is split-halves throughout, and there is no expert parallelism. `nanosarvam/README.md` has the full list and the module-to-line mapping.

What the small-scale runs say about each choice

Every row is a measurement in `docs/RESULTS.md`; the section numbers below refer to that file. Read the deltas as rankings, not as absolute quality — these are 17M–386M models.

design choice (above)	what nanosarvam measured	RESULTS
MLA vs GQA (§3)	At matched steps MLA wins by 0.034 bits/byte, as $2\times$ the active parameters should. At matched wall clock GQA wins outright, and on Hindi MLA's per-step edge vanished. The same trade the two real models make, landing the same way round.	\$1, \$2
KV cache (§3)	Both caches implemented and held to the uncached path token-for-token, including after the window fills. MLA caches <code>kv_lora_rank + qk_rope_head_dim</code> per token regardless of head count. The cache is worth $\sim 1.35\times$ at decode; the rest is $\sim 1,000$ kernel launches per token (33 tok/s at 386M).	\$9
YaRN vs a wide rope_theta (§4)	Reproduced exactly. The $\theta=1e4$ preset collapses at $4\times$ its training length without YaRN (loss $1.54 \rightarrow 3.03$ across the window) and is flat with it ($1.68 \rightarrow 1.71$). The $\theta=1e6$ preset extrapolates gently on its own; YaRN is a small net loss at 17M and a gain at 386M (-0.08 at $4\times$), where slow-frequency heads carry more. The temperature term has to reach the attention scale, not the tables — a GQA bug did exactly what trap 1 in §4 warns of and was caught by the position-bucketed eval.	\$3, \$15
Sigmoid + bias balancing (§5)	Works, and never enters the loss. Imbalance <i>rises</i> before it falls — $3\text{--}8\times$ in the first 100 steps, $1.1\text{--}1.4\times$ by the end — because a $\pm 1e\text{--}3$ step needs time to overtake the router's early drift. The <code>sign()</code> rule oscillates around its fixed point rather than converging (period 2, measured), and raising the rate $10\times$ makes balance <i>worse</i> ($2.59\times$). With 24 experts the floor is $\sim 1.8\times$ and costs no measurable loss.	\$5, \$16
Group-limited routing (§5)	4 groups keep 2 makes balancing slower (peak $7\times$) but not worse: it finishes lowest of the three presets at $1.13\times$. On Hindi it damped collapse rather than amplifying it, against the prediction going in. The config rejects a top-k larger than the surviving groups can supply — the footgun the 105B's <code>getattr</code> defaults leave open.	\$1, \$2
Shared expert (§2)	Not earning its 22M parameters at 386M on this data: removing it <i>improved</i> the benchmark by 0.003 bits/byte. Kept in both real models; at 128 experts and 16T tokens the trade may differ.	\$10
Fine-grained experts (§2)	24×192 top-6 against 12×384 top-3 at the same active FLOPs: -0.008 bits/byte, the largest single architecture move, at $1.57\times$ the wall clock from doubled launch count. Combined with no shared expert, top-4 and the window: the same loss at 76% of the active compute. DeepSeek-V3's argument, reproduced.	\$10, \$12
Sliding window on the lower layers (§7)	The dead <code>max_window_layers</code> field, made real: a 256 window on 10 of 20 layers costs nothing at the 512 training length and extrapolates <i>better</i> to 2048 (5.37 vs 5.44 naive), because half the layers never meet an unseen phase. The cheapest KV-cache reduction available.	\$10, \$15
Expert dispatch (§5)	The <code>argsort</code> dispatch as one grouped GEMM per projection (<code>F.grouped_mm</code>): $1.48\times$ faster than per-expert GEMMs on a GPU, one Dynamo graph where <code>nonzero</code> gives 8 breaks.	\$9
A training path (§6)	Trains, with differentiable <code>index_select</code> dispatch; gradient checkpointing takes effect (6.34 GB against 15.75 GB at 353M) instead of silently doing nothing.	\$6
The tokenizer (§9)	16k byte-level BPE, 1.34 tokens/word, -0.162 bits/byte at matched compute. More than $20\times$ parameters.	\$7
Data over architecture	Best model: the combo configuration on Wikipedia + Wikisource for 36k steps, 0.588 / 0.648 bits/byte on the two benchmarks. Mixing in a second register was worth 0.042 averaged over both; $3\times$ steps on mostly-unseen data 0.089; doubling the steps again, warm-restarted from the finished run, another 0.030.	\$17, \$18, \$21

design choice (above)	what nanosarvam measured	RESULTS
Accuracy, not just compression	On a 600-item Hindi cloze golden set built from the held-out text, the best model picks the right word from four 85.8% of the time (chance 25%) and reproduces the exact word 11.2%; the Wikipedia-only model 76.7% / 6.3%, with the whole gap on literature. Fluent, not factual: every date and figure in its samples is invented.	\$19
Post-training (§1: SFT then GRPO)	The SFT half, at nano scale: 56k question-answer pairs mined from the corpus's own lead sentences, loss on answers only, 15–20 minutes. The model learns the format completely (names the subject asked about 86% of the time, from 12%) and learns few new facts (answer-recognition 43% → 51% against a 25% floor). Doubling the pretraining underneath it moved compression by 0.03 bits/byte and recognition by one point: instruction tuning reveals what pretraining put in, and what pretraining puts in about any one subject is bounded by how often the corpus mentions it.	\$20, \$21

The ledger, largest lever first: tokenizer 0.162, more steps on new data 0.089, 20× scale 0.110 (on one benchmark), corpus cleaning 0.054, a second register 0.042, the training recipe 0.022, fine-grained experts 0.008, 3.3× unique data at fixed compute 0.007. Every mechanism in \$2–\$5 is real and reproduces at nano scale; none of them is within an order of magnitude of the tokenizer and the data. That is consistent with what §1 says about where Sarvam's own contribution lies.

Sources

Downloaded configs and modelling code ([sarvam/](#)); [sarvam-105b](#) and [sarvam-30b](#) model cards; [Sarvam's release blog](#); [Sarvam docs](#); [TechCrunch coverage](#).

Experiment log

Source: docs/RESULTS.md

Everything measured in this project, with the setup needed to reproduce it.

Sections 1–5 ran on **CPU** (torch 2.14.0+cpu) — the CUDA-capable GPU on the machine went unnoticed at first because the CPU-only torch wheel had been installed, and `torch.cuda.is_available()` dutifully said False. Read those wall-clock numbers as CPU numbers; comparisons between them are valid because they share a machine. Section 6 onward runs on the GPU (RTX 5050 8 GB, torch 2.14.0+cu130, bf16 autocast, fused AdamW).

Read the numbers as *relative* evidence about architecture choices, not as absolute quality. These are 17M and 47M parameter models.

The two presets

	nano_30b	nano_105b
attention	GQA, 6 q-heads / 2 kv-heads	MLA (kv_lora_rank 48)
layers	8	12
total params	17.44 M	47.39 M
active per token	6.60 M (37.9%)	13.33 M (28.1%)
experts	16, top-2, +1 shared	16, top-2, +1 shared
routing	n_group=1 (ungrouped)	n_group=4, topk_group=2
RoPE	theta=1e6, no scaling	theta=1e4, YaRN-ready

Both mirror their namesake's *ratios* (`moe_intermediate_size` = hidden/4 vs hidden/2, dense FFN = 2× vs 4× hidden), not just the name. Sparsity is milder than the originals: 16 experts top-2 is 12.5%, where the real models run 6/128 and 8/128 (~5%). Top-1 at this scale would be degenerate.

Parameter counts were derived analytically before the code ran and match `model.num_params()` exactly.

1. English (tinyshakespeare, 1.12 MB)

`--block-size 256 --batch-size 16`, identical seed and data order.

	GQA, 1000 steps	GQA, 600 steps	MLA, 600 steps
val loss	1.5190	1.5701	1.5467
bits/byte	2.192	2.265	2.231
wall clock	1345 s	921 s	1781 s

	GQA, 1000 steps	GQA, 600 steps	MLA, 600 steps
throughput	3000 tok/s	2692 tok/s	1393 tok/s
imbalance peak → final	5.19× → 1.16×	3.35× → 1.18×	7.02× → 1.13×

At matched steps (600), MLA wins — 1.5467 vs 1.5701, worth 0.034 bits/byte. It should: 2× the active parameters.

At matched wall clock, GQA wins outright. The 1000-step GQA run reaches a *better* loss (1.5190) in *less* time (1345 s) than MLA needs for 1.5467. Two extra active parameters bought 0.023 nats per step and cost 1.9× the time per step.

A first comparison at 1000 vs 600 steps was confounded by differing cosine schedules — at step 350 the longer run was still at lr 2.52e-4 while the shorter had decayed to 1.65e-4. The 600-step GQA run above was added specifically to remove that. Steps 0 and 100 are bit-identical across the two GQA runs (warmup overlaps), confirming determinism.

2. Hindi

Corpus from `fetch_hindi.py`, which streams the Hindi Wikipedia bz2 dump and stops early — **12.59 MB of clean prose from 3.1 MB downloaded (1% of the 240 MB dump)**, 450 articles, 73.8% Devanagari. The API route was abandoned: Wikipedia returns HTTP 429 within a few requests.

`--block-size 512 --batch-size 8`. The doubled block size is deliberate: Hindi runs at **2.48 bytes per character**, so a 256-byte window is only ~103 Devanagari characters — barely a sentence. Same tokens per step, twice the linguistic context.

	GQA 1000	MLA 1000	GQA 2000
val loss	0.7373	0.7450	0.6734
bits/byte	1.064	1.075	0.972
bits/char	2.64	2.67	2.41
wall clock	1859 s	3221 s	3345 s
imbalance peak → final	7.80× → 1.25×	5.41× → 1.34×	→ 1.23×

MLA's per-step advantage did not survive the language change. It won by 0.034 bits/byte on Shakespeare and *loses* by 0.011 on Hindi, with 2× the active parameters and 1.7× the wall clock. Most likely neither model is capacity-limited here: 1000 steps covers 4.1M tokens of an 11.3M-token split, so extra parameters have nothing to bite on. tinyshakespeare's 1 MB was small enough that the bigger model's fit mattered.

Group-limited routing damped rather than amplified collapse. Prediction going in was that Devanagari's clustered byte statistics would make the 105B's group constraint collapse harder. It started *better* balanced (5.41× vs 7.80×) and merely converged more slowly. Wrong prediction, recorded as such.

Doubling steps bought 0.092 bits/byte (1.064 → 0.972), inside the 0.05–0.12 predicted from log-linear scaling, and crossed below 1 bit/byte. Mid-run eval at step 999 of the 2000-step run was 0.7450 — essentially identical to the 1000-step run's final 0.7373 despite a 6× higher LR at that moment — so the gain came from the extra steps plus the anneal, not a lucky schedule.

The tokenizer cost, measured

A byte-level model on Hindi pays 2.48 tokens per character where English pays 1. The trained model learned UTF-8 unaided: over 1512 generated bytes, **zero decode errors**, 79.1% Devanagari (corpus 73.8%), 2.58 bytes/char (corpus 2.48). It inferred that Devanagari costs three bytes and which continuation bytes are legal, purely from byte statistics, before learning any Hindi.

bits/byte is misleading across scripts:

	bits/byte	bytes/char	bits/char
Hindi	0.972	2.48	2.41
English	2.192	1.00	2.19

Hindi looks twice as good per byte, but UTF-8 continuation bytes are nearly free to predict and drag the average down. Per character Hindi is *harder*. Always report bits/char when comparing across scripts.

This is the fertility argument made concrete: every Devanagari character costs 2.48 positions of context, KV cache and latency. Sarvam's tokenizer reports 1.4–2.1 tokens *per word*; bytes are ~15–20 per word.

Samples

1000 steps, prompt भारत एक:

भारत एक राजीवराष्ट्रीय कांग्रेस के तर्काण करते हैं, जिसके तौर फैलिक भाषाओं के लोगों का एक बाह्य...

2000 steps, prompt मुंबई शहर:

मुंबई शहर की पुराततिमान विद्वानों में संख्या सम्मिलित हैं। उच्च न्यायालय संस्कृत में प्राचीन काल की मृत्यु के गान में सौप मध्य कोश नाम किया है।

Clear local improvement — real multi-word constructions (उच्च न्यायालय, प्राचीन काल, के अनुसार), the को को stutter gone, clause structure holding longer. Still semantically incoherent; 17M parameters. Occasional `ref` /`ref` leakage is the wikitext scrubber, not the model.

The 17M model on the big corpus: stopped at step 999

A 6000-step run of the same 17M `nano_30b` on a **107.7 MB** corpus (`fetch_hindi.py --mb 100`, ~26 MB downloaded, 11% of the dump) — 24.6M tokens against a 96.9 MB split, i.e. **0.25 epochs**, the first run where no text would repeat. Prior runs did 0.36 and 2.9 epochs. It was **killed at step 999 of 6000** when the GPU turned up (\$6), and never resumed: the 353M took over that corpus instead. What it left behind, `ckpt_hindi_big.pt` (and a `.last` that `--resume` would continue from):

	ckpt_hindi_2k (12.6 MB, 2000 steps)	ckpt_hindi_big (107.7 MB, 999 of 6000 steps)
own val	0.6734	0.7486 (eval @ 499: 0.8598)
bits/byte (own split)	0.972	1.080

Not comparable: different val splits (\$8 is where that lesson was learned) and a cosine schedule still near its peak at step 999 of 6000. The CPU question it was asking — does a 17M model gain from text that never repeats — was answered at 353M instead (\$6, \$7, \$11).

3. Does YaRN earn its place?

Both Shakespeare checkpoints were trained at `block_size=256` with plain RoPE. Evaluated at $2\times$ and $4\times$ that, either naively or with the rotary rebuilt for YaRN (`factor = length/256`). Same weights, **zero fine-tuning**. Reproduce with `python eval_context.py --ckpt ckpt_105b.pt`.

Loss by position inside a 1024-byte window:

positions	105b naive	105b YaRN	30b naive	30b YaRN
0–256 (<i>in-distribution</i>)	1.5421	1.6802	1.5076	1.6808
256–512	1.7924	1.6989	1.6046	1.7052
512–768	2.8697	1.7034	1.6861	1.6898
768–1024	3.0342	1.7131	1.8616	1.6882

For the MLA preset YaRN is the difference between working and not. Naive extension collapses $1.54 \rightarrow 3.03$ as you walk out the window. With YaRN the curve is nearly flat, $1.68 \rightarrow 1.71$ across a $4\times$ extension.

For the GQA preset YaRN is a net loss — but a close one. `rope_theta=1e6` already extrapolates gently ($1.51 \rightarrow 1.86$), so there is less for YaRN to repair, and on average the repair does not pay for itself: $+0.0589$ at 512, $+0.0260$ at 1024. What it buys is the same flat curve the MLA preset gets — it never leaves $1.68\text{--}1.71$ across the window — so it ties in the third quarter (1.6898 vs 1.6861) and wins outright in the fourth (1.6882 vs 1.8616).

YaRN's price is visible in both: at 0–256 it is *worse* than naive (1.68 vs 1.54 ; 1.68 vs 1.51). You trade accuracy on the range you trained for against not falling apart beyond it. A single averaged number hides this entirely — the position-bucketed view is what exposes it.

Correction. The GQA column was regenerated after a fix. `GQAttention` set `scale = head_dim**-0.5` and never applied YaRN's logit-temperature term (`mscale**2`, 1.296 at factor 4), which `MLAttention` had carried since it was written — so the earlier GQA row measured the band blend alone while the prose credited both halves, and quoted $1.874\times$, which is `mscale**2` at the real 105B's factor 40, not at the factor 4 measured here. With the term applied, YaRN's far-end win on GQA roughly doubles (1.6882 vs the previous 1.7854 , against naive 1.8616) and its in-distribution cost grows (1.6808 vs 1.6482). The verdict does not change. The MLA columns are untouched by the fix and reproduce to the fourth decimal, which is what says the harness itself is stable.

The two presets independently reproduce **why Sarvam chose oppositely**: 30B at `rope_theta=8e6` with `rope_scaling: null`, 105B at `rope_theta=1e4` with YaRN factor 40. Each strategy wins on its own model.

4. MoE dispatch: sparse vs dense

	eager, nano_30b	eager, nano_105b	Dynamo	equivalence
sparse (default)	574 ms/step	1407 ms/step	9 graphs, 8 breaks	—
dense	1350 ms/step	2955 ms/step	1 graph, 0 breaks	bit-identical

Sparse dispatch uses `nonzero`, whose output shape is data-dependent, so Dynamo cannot capture it. Setting `capture_dynamic_output_shape_ops` does not help, and neither does removing the `if sel.numel() == 0` guard — both were tested. `dense` runs every expert on every token and combines through a one-hot weight matrix: static shapes, `num_experts/top_k` times the expert FLOPs.

Sparse stays the default (it is what a real MoE does, and it wins in eager); use `--moe-impl dense` when you want `--compile` to mean something. Graph-break counts are asserted in tests **in both directions**, so if a future torch stops breaking on `nonzero` the test fails rather than the docs silently rotting.

5. Load balancing behaviour

The aux-loss-free bias is the one mechanism the released inference code shows being *consumed* but never *written* — it is training-time only. Findings from implementing it:

- **Imbalance rises before it falls, in every run.** At `rate=1e-3` the bias moves ± 0.05 in 50 steps against sigmoid scores spanning (0,1), so the router collapses toward favourites faster than the correction pushes back. Past $\sim$ step 100 the accumulated bias overtakes it. Peaks observed: $5.19\times$, $7.02\times$, $7.80\times$, $5.41\times$; finals: $1.13\times$ – $1.34\times$. **Do not read the early climb as a broken balancer.**
- **The rule oscillates and never settles.** `sign()` applies a full-size step however small the error. Measured ping-pong: $1.188\times \leftrightarrow 1.734\times$, period 2, at `rate=0.05` on a frozen batch. Inherent to DeepSeek-V3's rule; harmless in real training where the rate is $50\times$ smaller and batch variation averages it.
- **Recovery time is arithmetic.** An overloaded expert's bias falls by `rate` while a starved one rises by `rate`, so a skew of S closes at $2\cdot\text{rate}$ per step and needs $\sim S/(2\cdot\text{rate})$ **steps** before routing can change at all. Measured: skew 3.0 at rate 0.01 shows *zero* movement in 120 steps. Check the budget before raising `bias_update_rate` because imbalance looks stuck.

6. Scaling to the GPU: sizing `nano_350m`

Target was "500M on the 8 GB RTX 5050". The arithmetic said Muon would make 443M fit; the measurement said otherwise. All rows: batch 8 $\times$ block 512, bf16 autocast, gradient checkpointing on unless stated.

total params	optimizer	ms/step	tok/s	peak GB	verdict
443M	AdamW, no ckpt	31573	130	15.75	WDDM spill, thrashing
443M	AdamW	2716	1508	9.74	still spills
443M	Muon	6997	585	7.99	fits, but $2.6\times$ slower
353M	AdamW	827	4955	6.34	the preset
281M	AdamW	696	5887	5.15	headroom

Two findings worth keeping:

- **Windows WDDM hides OOM.** Exceeding VRAM spills to system RAM instead of raising an error — the 443M/no-ckpt row "works" at $12\times$ slowdown. On Linux those rows would crash, which is kinder.
- **Muon is the wrong tool for fine-grained MoE on one small GPU.** It converges faster per step (3.12 vs 3.50 at step 19 of a smoke run) and stores one momentum buffer instead of AdamW's two moments (12

vs 16 bytes/param) — but Newton–Schulz runs per-tensor, and this model has 383 small 2D matrices (12 experts × 3 matrices × 19 layers + attention), so launch overhead makes it **2.6× slower per step**. Muon's wins come on dense models with few large matrices; use `--optimizer muon` here only when memory, not time, is the constraint. Implementation in `muon.py`.

Final preset: `nano_350m` — 353M total / 152M active, 20 layers, hidden 1024, 12 experts top-3 + 1 shared, GQA 16q/4kv, `rope_theta 1e6`. Requires `--grad-ckpt` (15.75 GB without, 6.34 GB with).

353M on Hindi (GPU)

6000 steps on the 107.7 MB corpus (0.25 epochs — no repeated text), batch 8 × block 512, bf16 autocast, fused AdamW, `--grad-ckpt`, lr 6e-4, 93 minutes.

```
eval @ 499 | val 0.8776
eval @ 1499 | val 0.7093      <- near the 17M's *final* result, a quarter in
eval @ 2999 | val 0.6623      <- past it at the halfway mark
eval @ 4999 | val 0.6228
eval @ 5499 | val 0.5975 | bits/byte 0.862  <- best (saved)
eval @ 5999 | val 0.6072      <- noise; best-val checkpointing earned its keep
```

	17M ckpt_hindi_2k (CPU)	353M ckpt_hindi_350m (GPU)
total / active	17.4M / 6.6M	353.3M / 151.5M
corpus / epochs	12.6 MB / 2.9	107.7 MB / 0.25
val loss	0.6734	0.5975
bits/byte	0.972	0.862
bits/char	2.41	2.14
throughput	2455 tok/s	4400 tok/s
wall clock	56 min	93 min

20× the parameters and 8.6× the data bought **0.11 bits/byte** (0.27 bits/char) — real, but a reminder that loss compresses logarithmically; the sample quality moved more than the number suggests. The 353M output holds idiomatic multi-clause constructions (के रूप में जाना जाता है, consistent postpositional case) across full sentences where the 17M model managed fragments. Still not factual — that plausibly needs another order of magnitude in both parameters and tokens.

Load balancing at 353M behaved like every other run: 3.97× early, 1.28–1.40× at the end, bf16 notwithstanding — the fp32 router and QK-norm did their jobs; no instability anywhere in the curve.

7. The tokenizer experiment: bytes vs BPE at matched compute

The largest single result in this log. `bpe.py` is a from-scratch byte-level BPE (stdlib only): 16,384-token vocab trained on the 12.6 MB corpus in **10 seconds** (unique-chunk weighting + incremental pair counts + lazy max-heap), exact roundtrip by construction, byte fallback for unseen scripts.

Fertility, measured on the held-out 107 MB corpus:

tokenizer	bytes/token	tokens/word	text in a 512-token window
bytes	1.00	~15–20	~206 Devanagari chars

tokenizer	bytes/token	tokens/word	text in a 512-token window
hi16k BPE	9.64	1.34	~2,000 chars

1.34 tokens/word sits inside the 1.4–2.1 Sarvam reports for their 262k-token, 22-language tokenizer — achievable monolingually with 16k tokens and 10 seconds of training. Then two runs differing **only in tokenizer**: same 353M trunk (the BPE model is 386M total — the extra 33M is the bigger embedding+head), 6000 steps, batch 8×512 , same schedule, same GPU.

	byte ckpt_hindi_350m	BPE ckpt_hindi_bpe
vocab	259	16,384
val loss (nats/token)	0.5975	4.6795
bits/byte	0.862	0.700
bits/char	2.14	1.74
text seen per step	~4 KB	~39 KB
corpus epochs	0.25	2.4
wall clock	93 min	100 min

Raw losses are incomparable across vocabularies (0.60 vs 4.68 — predicting one of 16,384 is a harder per-token task than one of 259); bits/byte is the honest axis, and the `.bin` metadata carries the conversion. **The tokenizer bought 0.162 bits/byte at identical compute — more than the 20× parameter scale-up bought (0.110)**. Mechanism: every training step witnessed $9.64\times$ more Hindi.

Two caveats, stated rather than hidden. The BPE run cycled its corpus $2.4\times$ where the byte run never repeated — that asymmetry is inherent to the comparison (a tokenizer compresses the corpus too), and val loss was still falling at the end ($0.717 \rightarrow 0.700$ over the last 1500 steps), so repetition had not begun to bite. And the byte model gets one structural consolation: it can never emit an out-of-vocabulary word, while the BPE model's rare-word tail is thinner.

Sample quality moved more than either loss number. The BPE model produces paragraph-length, topic-coherent Wikipedia-register Hindi — a Mumbai prompt yields a sustained tourism description across eight sentences — where the byte 353M managed correct single sentences and the 17M managed fragments. Still factually unreliable (invented capitals, "पाकिस्तान और पाकिस्तान के बीच"), and it inherits the corpus's `ref` leakage verbatim.

Generation is also $\sim 9.6\times$ cheaper per character: 120 generated tokens now yield $\sim 1,150$ bytes of text where the byte model needed $\sim 1,200$ tokens for the same.

All four Hindi models:

	params	tokenizer	device	bits/byte	bits/char
ckpt_hindi_2k	17M	bytes	CPU	0.972	2.41
ckpt_hindi_105b	47M	bytes	CPU	1.075*	2.67*
ckpt_hindi_350m	353M	bytes	GPU	0.862	2.14
ckpt_hindi_bpe	386M	hi16k	GPU	0.700	1.74

*1000 steps, not directly comparable to the 2000/6000-step rows.

8. Corpus cleaning: v2

The v1 BPE model had *learned the corpus's markup leakage as a genre feature* — `ref name=` fragments mid-sentence, and one prompt (क्रिकेट) collapsing entirely into hallucinated `gallery/File:` blocks. Two fixes in `fetch_hindi.py`: entity-decode **before** tag-stripping (the dump stores tags as `<ref>`, so v1 stripped tags first and left the unmasked text behind), and `--keep-prob` sampling across the **whole** dump instead of taking the front 11% (which had monuments but hardly any cricket).

v2 corpus: 192.4 MB from 100% of the dump at keep-prob 0.3. Per MB: `ref name` 116.7 → **0.01**, `ref /ref` 182.7 → **0.00**, `.jpg` 19.4 → 2.6; क्रिकेट mentions 2.4×. Tokenizer retrained on the clean text improved slightly (1.30 tok/word, 9.79 B/tok). Same 6000-step recipe, ~1.25 epochs.

Fair comparison requires shared text — each run's own val split is different text, so own-val numbers (0.700 dirty, 0.631 clean) don't compare. On an identical 3.8 MB held-out slice, each model using its own tokenizer:

	bits/byte on shared text
v1 (dirty corpus)	0.6631
v2 (clean corpus)	0.6087

Cleaning bought 0.054 bits/byte at identical compute — about half what the tokenizer bought, and for ~30 minutes of scrubber work.

Qualitatively: the v1 failure modes are gone (no `ref` fragments anywhere; the क्रिकेट prompt now degrades gracefully into prose instead of file-markup) — but cleaning is whack-a-mole, and removing the dominant artifact promoted the next ones. v2 samples surface **numeric citation brackets** (`[3]` `[5]` `[1]`), present in the source text and untouched by the scrubber, likely unmasked by the same entity decoding) and occasional English pseudo-title runs from the broader article mix. Also spotted: `----` (wikitext horizontal rule) survives. Each is a one-line scrubber fix for the next data pass; the lesson worth keeping is that **the model is a magnifying glass for whatever is left in the data** — 0.01% contamination is invisible in loss and visible in generation.

Operational note: the run was interrupted twice and recovered both times via `--resume`. The second recovery exposed a real bug — the resume path loaded the 4.6 GB optimizer checkpoint onto the GPU with `map_location=args.device` and kept it referenced for the life of the run, forcing permanent WDDM spill (210 tok/s instead of ~4,400, GPU at 24 W). Fix: load to CPU, let `Optimizer.load_state_dict` place states, `del` the blob. A kill mid-`torch.save` also truncated the best-val file once; the `.last` file restored it. Both behaviours are why the two-file checkpoint scheme exists.

Reproducing

```
cd nanosarvam
python train.py --preset 30b --steps 1000 --batch-size 16 --block-size 256
python train.py --preset 105b --data data/hindi.txt --block-size 512 \
    --batch-size 8 --steps 1000
python eval_context.py --ckpt ckpt_105b.pt
python test_model.py
```

Seed is 1337 everywhere and runs are deterministic on identical hardware and torch version.

9. The v4 recipe, A/B at matched compute, on a shared benchmark

Hardware note: sections 6–8 ran on an RTX 5050 8 GB; the machine now carries an **RTX 5070 Ti 16 GB** (torch 2.11.0+cu128). Wall-clock numbers here are not comparable to earlier sections; everything else is.

The benchmark

Own-val numbers cannot be compared across corpora or tokenizers (§8). So: `make_bench.py` builds one fixed held-out file from a *differently seeded* sample of the dump (`--seed 2024 --keep-prob 0.05`) and drops every document whose Devanagari-only signature (three 80-letter windows) appears in **any** corpus a checkpoint here trained on. The first cut used a plain 200-character substring probe, which missed articles the v1 scrubber had mangled — and `ckpt_hindi_bpe` (v1 corpus) scored 0.637 bits/byte on it, "best of all". With the script-only signature that number is 0.734, worst of the BPE models. **A benchmark leak of 11% of documents moved a model by 0.1 bits/byte**; 1,225 of 2,572 candidate documents had to go. Final file: 1,347 docs, 13.61 MB, 5.41 M chars. `eval_bench.py` scores in non-overlapping `block_size` windows, plain `encode` (no EOS, so older models are not penalised), each checkpoint with its own tokenizer.

The A/B

Same trunk (`nano_350m`, 386M with the 16k head), same corpus *text* (`hindi_v4.txt`, 194 MB, 21,897 articles with titles), same tokenizer (`hi16k_v4`), same seed, 6000 steps $\times$ 8 $\times$ 512, lr 6e-4, `--grad-ckpt`, `--moe-impl grouped`, bf16.

	A: old recipe	B: v4 recipe
document boundaries	none (articles run together)	EOS after every article
sampling	with replacement (random offsets)	<code>WindowSampler</code> , no replacement, 1.41 epochs
router regularisers	—	seq-balance 1e-4, z-loss 1e-3
own val (nats/token)	4.5606	4.3728
final imbalance	1.46 $\times$	1.27$\times$
throughput	11.8k tok/s	11.0k tok/s (the regularisers cost ~7%)
wall clock	2080 s	2241 s
benchmark bits/byte	0.6878	0.6655
benchmark bits/char	1.7309	1.6747

–0.022 bits/byte at identical compute on identical text, scored without EOS (so B gets no credit for the boundary tokens it learned). For scale: the tokenizer bought 0.162, cleaning 0.054, 20 $\times$ parameters 0.110. This is a recipe change, not a data change, and it lands at ~40% of what cleaning did.

Own-val is *not* the fair axis here — B's split contains EOS tokens and document starts (the first tokens after an EOS are near-unpredictable, which is why B's *training* loss reads 0.4–0.5 higher than A's all run while its val reads lower). The benchmark is.

All eight, one axis

checkpoint	params	tokenizer	corpus	recipe	bits/byte	bits/char
<code>ckpt_hindi_2k</code>	17M	bytes	v1 12.6 MB	old	1.0711	2.6954

checkpoint	params	tokenizer	corpus	recipe	bits/byte	bits/char
ckpt_hindi_350m	353M	bytes	107 MB	old	0.9556	2.4046
ckpt_hindi_bpe	386M	hi16k	107 MB (dirty)	old	0.7342	1.8476
ckpt_hindi_v2	386M	hi16k_v2	v2 192 MB	old	0.6810	1.7136
ckpt_hindi_v3	386M	hi16k_v3	v3 192 MB	old	0.6933	1.7446
ckpt_hindi_v4_base	386M	hi16k_v4	v4 194 MB	old	0.6878	1.7309
ckpt_hindi_v4	386M	hi16k_v4	v4 194 MB	v4	0.6655	1.6747

Two things the single axis exposes. v2 → v3 → v4_base are the *same recipe* on three re-fetches of the same dump and land within 0.012 of each other — that is the corpus-to-corpus noise floor, and the v4 recipe's 0.022 clears it by 2×. And §8's "cleaning bought 0.054" survives (dirty 0.734 → clean 0.681), though the shared 3.8 MB slice it was measured on was itself fetched with the same seed as the training corpora and so was probably partly seen.

Samples, prompt मुंबई\n\n (title, blank line — the document format)

Old recipe, 250 tokens, never stops: a temple that becomes a temple inside a temple —

मुंबई के महाराजा के रूप में प्रसिद्ध है। इस कारण से मंदिर के दर्शन करने के लिए ... यह मंदिर शिव के अवतार के रूप में जाना जाता है और मंदिर में एक मंदिर के साथ एक मंदिर के रूप में भी पूजा जाता है। ...

v4 recipe, **61 tokens, stopped at EOS**: an article-shaped paragraph that opens on the subject, gives (invented) population figures, closes on a summary sentence, and ends —

मुंबई के रूप में जाना जाता है और इसमें 4000 तक के साथ 4000 से अधिक लोग शामिल हैं साथ ही साथ 1400 से अधिक लोगों को भी शामिल किया गया है। भारत में 49% भारत में रहते हैं और सभी क्षेत्रों में रहते हैं। यह भारत में सबसे अधिक आबादी वाले राज्यों में से एक है।

Still not factual — the numbers are made up, and it calls Mumbai a state — but it now knows what a document is: where one starts, what its first sentence does, and when to stop. That is what the EOS bought, and no loss number shows it.

Dispatch and decoding, same session

--moe-impl grouped vs sparse on this trunk, 40-step probes: **11.9k vs 8.0k tok/s** (1.48×), and 328 ms/step flat against 330–560 ms jittering — sparse does a nonzero and a .numel() per expert per layer, a device sync each. Decoding ckpt_hindi_v3, 300 tokens, batch 1: sparse 16 tok/s with or without the KV cache; grouped 21 uncached → 28 cached → **33** with the expert stack cached in eval mode. ~1,000 kernel launches per token is the ceiling on this machine (no host compiler for Inductor or CUDA graphs), and it is a serving problem, not a model problem.

10. Architecture ablations on the v4 recipe

Five runs, everything identical to §9's run B except one --cfg override each, at a **2000-step** budget (a third of §9's), scored on the shared benchmark. Directions at 2000 steps are usually the directions at 6000;

magnitudes are not, so read the deltas as rankings.

	params (total / active)	own val	bench bits/byte	Δ vs baseline	wall
baseline (12×384, top-3, +1 shared)	386M / 185M	5.2973	0.7970	—	770 s
no shared expert	364M / 163M	5.2773	0.7937	−0.0033	779 s
fine-grained 24×192, top-6	375M / 185M	5.2415	0.7891	−0.0079	1209 s
top-2 instead of top-3	386M / 163M	5.2812	0.7941	−0.0029	1099 s
sliding window 256 on layers 0–9	386M / 185M	5.2803	0.7953	−0.0017	894 s

Every ablation beat the baseline, which says the preset is mis-tuned rather than that any single idea is magic. Three of them beat it with *fewer* active parameters (no shared expert, top-2) or the same (window), so the "loss vs FLOPs" story is better than the table looks:

- **Fine-grained experts win outright** — same active FLOPs, −0.008 bits/byte, the largest single move here. Which is DeepSeek-V3's argument (many small experts, higher combinatorial capacity) reproduced at 386M. The price is launch count: 24 experts is twice the kernels per layer, 1.57× the wall clock in this launch-bound regime, and on a compiled stack that tax mostly disappears.
- **The shared expert is not earning its 22M parameters** at this scale on this data. Drop it and the model gets *better*. Sarvam keeps one in both real models; at 128 experts and 4T tokens the trade may differ.
- **Top-2 ≥ top-3**: 12% fewer active parameters, slightly better loss. The aux-loss-free balancer keeps top-2 at 1.3× imbalance without trouble.
- **Windowing half the layers is free** on 512-token training, as it should be — a 256 window at block 512 removes almost nothing a layer-0 head wants — and it is the cheapest KV cache reduction available. It needs a long-context eval (`eval_context.py`) to say anything about what it costs beyond 512.

Wall-clock varies ±20% run to run on this box (WDDM, background load); the kernel-count argument for fine-grained experts is sound, the exact ratio is not.

A combined preset — 24×192 experts, top-4, no shared expert, window on the lower half — is the obvious next run; it was not made here because each ablation was run against the baseline alone, and interactions are exactly what that design cannot see.

11. The data lever: the whole dump at the same compute

`fetch_hindi.py --keep-prob 1.0` on the full dump: 72,467 articles, 648 MB. Every benchmark article removed by title (1,347) plus 47 near-duplicates by signature, leaving **633.7 MB, 71,073 documents, 62.9M tokens** with a tokenizer trained on it (`hi16k_full`, 10.07 B/tok). Then §9's run B exactly — same 6000 × 8 × 512, same recipe — so the only change is 3.3× the unique text: **0.43 epochs instead of 1.41**.

	ckpt_hindi_v4	ckpt_hindi_full
unique tokens / epochs	19.1M / 1.41	62.9M / 0.43
own val (own split)	4.3728	4.3259
bench bits/byte	0.6655	0.6586
bench bits/char	1.6747	1.6572
wall clock	2241 s	2855 s

–**0.007 bits/byte**. Real, and small — a third of the recipe change, a seventh of the tokenizer. This is what the repetition literature predicts: below ~4 epochs, repeated data is nearly as good as fresh data, so a corpus that was only being cycled 1.4× had little to gain from not being cycled. The model is compute-bound at this budget, not data-bound. More data starts paying when steps go up with it; at 6000 steps the 62.9M tokens are 57% unseen.

The sample (मुंबई\n\n, 34 tokens, stopped at EOS) came out as a stub — a heading and a list of road and airport names — which the full dump has far more of than the 30% sample the earlier runs saw (the sampled corpora and this one share the 600-character floor, but the tail of short list-like articles is long). One draw at temperature 0.8 is not evidence; it is a reminder that "more data" changed the *mix*, and a corpus filter on prose density would be the next data-side lever.

Where this leaves the log

lever	bits/byte bought	section
16k BPE instead of bytes	0.162	\$7
20× parameters + 8.6× data	0.110	\$6
corpus cleaning	0.054	\$8
v4 recipe (documents, no-replacement sampling, regularisers)	0.022	\$9
fine-grained experts (2000-step ablation)	0.008	\$10
3.3× unique data at fixed compute	0.007	\$11

Best model on the shared benchmark: **ckpt_hindi_full**, **0.6586 bits/byte**, **1.657 bits/char**. The next order-of-magnitude move is not on this list; it is more steps on more data, which is a GPU-hours question, not an engineering one.

12. The ablation winners together

```
--cfg num_experts=24 moe_intermediate_size=192 num_experts_per_tok=4 num_shared_experts=0
sliding_window=256 max_window_layers=10, v4 recipe on the full corpus, 6000 steps — $11's
ckpt_hindi_full is the control.
```

	ckpt_hindi_full	ckpt_hindi_combo
experts	12×384, top-3, +1 shared	24×192, top-4, no shared, window 256 on 10/20 layers
params total / active	386M / 185M	364M / 140M (–24%)
own val (same split)	4.3259	4.3334
bench bits/byte	0.6586	0.6595
final imbalance	1.42×	1.76×
wall clock	2855 s	2426 s (–15%)

Same loss, three-quarters of the active compute. The three ablations that each won at 2000 steps did not add up to a lower loss at 6000; they added up to a *cheaper* model at the same loss — +0.001 bits/byte, inside the noise, for 24% fewer active parameters and 15% less wall clock even with twice the expert launches. Two readings, both probably true: at 2000 steps the wins were partly "learns faster early", which a

3× longer schedule erases; and top-4 of 24 (17%) is a coarser selection than top-6 of 24 (25%), so some of the fine-grained gain was spent buying the FLOP cut.

Balancing got harder: 24 experts finish at 1.76× against 1.42× — the bias rule has twice the slots to equalise with the same step size. Raising `bias_update_rate` for fine-grained configs is the obvious knob, untested.

Samples (`--seed 7`, 200 tokens):

- मुंबई\n\n — a "how to reach" section: सड़क मार्ग / हवाई मार्ग paragraphs about highways and stations, fluent and repetitive, never stopping.
- भारतीय क्रिकेट टीम\n\n — two clean sentences (appointed captain, added to the team on 7 December 2019) and EOS at 23 tokens.
- हिन्दी साहित्य\n\n — one sentence, then an award list: साहित्य अकादमी पुरस्कार –2012, –2002, –2009 ... EOS after 186 tokens.

Two of three drift into the list register within a paragraph. That is the corpus, not the architecture — hence §13.

13. Prose filtering: a clean negative

`filter_prose.py` on the full corpus — drop lines under 40 characters that do not end a sentence, then drop articles with under 40% sentence-ended lines — keeps 89% of the bytes (63,017 of 71,073 documents, 54.7M tokens with the same `hi16k_full` tokenizer). Same recipe and budget as `ckpt_hindi_full`.

	<code>ckpt_hindi_full</code>	<code>ckpt_hindi_prose</code>
tokens / epochs	62.9M / 0.43	54.7M / 0.50
own val	4.3259	4.3278 (different split)
Wikipedia bench bits/byte	0.6586	0.6818
Wikisource bench bits/byte	0.8947	0.9172

Worse on both, by 0.023. The Wikipedia loss was predictable — the benchmark is unfiltered, a quarter of it list-shaped, and the model never saw a list — but the literature benchmark is pure prose and the prose model loses there too. So the lines the filter removed were carrying signal: headings and list items are where names, dates and entity types cluster, and a model that has seen fewer of them predicts them worse *in running prose as well*. The samples did come out prose-shaped (a film-release paragraph for मुंबई, two clean sentences and EOS for the cricket prompt) — the filter fixed the generation register and paid for it in loss. That is a real trade, but it is not the one the section-11 sample implied: the model's list habit is learned from data that is also teaching it Hindi.

14. Out of register: Hindi Wikisource

`fetch_hindi.py --dump ...hiwikisource...` gives 1,426 scanned book pages — Premchand's कर्मभूमि on the first two — 5.94 MB of literary Hindi that no checkpoint trained on, no de-duplication needed. Every 386M model:

checkpoint	corpus / recipe	Wikipedia	Wikisource	gap
<code>ckpt_hindi_bpe</code>	v1 dirty, old	0.7342	0.9408	0.207

checkpoint	corpus / recipe	Wikipedia	Wikisource	gap
ckpt_hindi_v2	v2, old	0.6810	0.9242	0.243
ckpt_hindi_v3	v3, old	0.6933	0.9350	0.242
ckpt_hindi_v4_base	v4, old	0.6878	0.9276	0.240
ckpt_hindi_v4	v4, v4 recipe	0.6655	0.9032	0.238
ckpt_hindi_full	full, v4 recipe	0.6586	0.8947	0.236
ckpt_hindi_prose	prose, v4 recipe	0.6818	0.9172	0.235
ckpt_hindi_combo	full, v4 + §12 preset	0.6595	0.8970	0.238

Two things. **The ranking transfers**: every gain measured on Wikipedia — recipe, data, the combined preset's parity — shows up at nearly the same size on a register none of the models saw, which is the evidence that the numbers in §9–12 are not benchmark-fitting. And **the register gap is flat at ~0.24 bits/byte** for every model regardless of corpus or recipe, i.e. it is a property of training on Wikipedia at all, not of how. Closing it means a different corpus mix, not a better recipe — which is the honest ceiling on what this log can reach with a single-source dataset.

15. What the sliding window costs past the training length

`eval_context.py --device cuda` on the two 2000-step ablations — the baseline and the window-256-on-layers-0–9 model — at 1×, 2× and 4× the 512-token training length, naive and with YaRN (`factor = length/512`):

context	baseline naive	baseline YaRN	window naive	window YaRN
512 (trained)	5.2782	—	5.2656	—
1024	5.3835	5.3693	5.3596	5.3554
2048	5.4428	5.3593	5.3746	5.3466

positions in a 2048 window	baseline naive	window naive	baseline YaRN	window YaRN
0–512	5.3211	5.3146	5.3433	5.3332
512–1024	5.2997	5.2646	5.2597	5.2459
1024–1536	5.5047	5.4187	5.3917	5.3824
1536–2048	5.6454	5.5007	5.4424	5.4247

The window costs nothing and extrapolates better. Half the layers never see a position beyond 256, so only the other half meet unseen phases at 2048; the windowed model degrades 5.27 → 5.37 naive where the baseline goes 5.28 → 5.44, and it still wins with YaRN on. The far-quarter gap is 0.145 nats. At 386M with `rope_theta=1e6` YaRN now *helps* (−0.08 at 4× for the baseline), unlike the 17M `nano_30b` in §3 where it was a net loss — the larger model's slow-frequency heads are doing more, so there is more to protect. This is the cheapest KV cache reduction available and it is free on quality; the §12 preset was right to keep it.

The ledger, closed

lever	bits/byte	
16k BPE instead of bytes	0.162	\$7
20× parameters + 8.6× data	0.110	\$6
corpus cleaning	0.054	\$8
v4 recipe (documents, no-replacement sampling, regularisers)	0.022	\$9
fine-grained experts (2000-step ablation)	0.008	\$10
3.3× unique data at fixed compute	0.007	\$11
\$12 preset: same loss at 76% of active compute	0.000 (−24% FLOPs)	\$12
prose filtering	−0.023	\$13
sliding window on the lower half	0 at 512, −0.07 at 2048	\$15

Best on both benchmarks: `ckpt_hindi_full` (0.6586 Wikipedia, 0.8947 Wikisource); `ckpt_hindi_combo` matches it at three-quarters of the active compute and is the model to build on.

16. `bias_update_rate` for fine-grained experts

\$12 flagged that 24 experts finish at 1.76× imbalance against 12 experts' 1.42×. Three rates on the combo preset, full corpus, 2000 steps each:

rate	own val	bench bits/byte	imbalance, mean of last 10 logs	range
1e-3 (default)	5.2348	0.7885	1.90×	1.58–4.88×
3e-3	5.2282	0.7870	1.88×	1.58–4.64×
1e-2	5.2279	0.7870	2.59×	2.02–4.47×

The imbalance is not costing loss. Tripling the rate buys 0.0015 bits/byte and moves the balance by nothing measurable; ten× buys the same 0.0015 and makes the balance *worse* on average (2.59×), because a `sign()` step of 0.01 against sigmoid scores in (0, 1) overshoots and the rule ping-pongs — \$5's oscillation, now visible at training scale. The 1.7–1.9× floor at 24 experts is what a fixed-step balancer settles to with twice the slots per token; a smaller model on more data has enough spare capacity that the imbalance is paid for in routing, not in perplexity. 3e-3 is marginally the right default for the fine-grained preset; the difference is inside the run-to-run noise of \$10.

17. Closing the register gap: Wikisource as training data

\$14 found a flat 0.24 bits/byte gap between Wikipedia and literature for every model, and said it was a corpus problem. Test: the whole Hindi Wikisource dump (154,843 scanned book pages, **689 MB — larger than Hindi Wikipedia**), the 1,426 benchmark pages removed by exact title, interleaved document-by-document with the full Wikipedia corpus so the val split is mixed too. New tokenizer (`hi16k_mixed`, 9.41 bytes/token — the older orthography costs a little fertility). Combo preset, same 6000-step budget: **140.6M tokens, 0.17 epochs**, half of them Wikipedia.

	ckpt_hindi_combo (Wikipedia only)	ckpt_hindi_mixed
Wikipedia bench	0.6595	0.7068 (+0.047)
Wikisource bench	0.8970	0.7660 (−0.131)
register gap	0.238	0.059
mean of the two	0.778	0.736
final imbalance	1.76×	1.56×

The gap is three-quarters closed, for a fifth of it back on Wikipedia. Half the Wikipedia tokens cost 0.047 there; the literature the model had never seen is worth 0.131 where it matters. Averaged over both registers it is the best model in the log by 0.042 — more than the recipe change and the data lever together — and it is the first result here that is not a refinement of "train on Wikipedia better".

The samples switch register by prompt. मुंबई\n\n becomes a political biography (born 15 May 1927, Congress, chief minister of Delhi, then of Uttar Pradesh — all invented, all in the encyclopaedic voice). उसने कहा\n\n — "he said" — becomes first-person narrative and *stops at EOS*: "मैं ऐसा ही करना चाहता हूँ कि मैं भी इस विश्वासके कारण ...", with Wikisource's pre-1950s orthography intact (विश्वासके, बारेमें, उद्देश्यको — postpositions joined to the noun). It learned the register and its spelling conventions from 0.17 epochs, which says how little of the Wikisource text it needed.

The cost side is real and should not be waved away: a Wikipedia-only deployment would take the 0.047. But nothing a user asks a Hindi model is Wikipedia-shaped, and the 0.24 gap was the number that said so.

The ledger, reopened and closed again

lever	bits/byte	
16k BPE instead of bytes	0.162	\$7
mixing in a second register (mean of both benchmarks)	0.042	\$17
20× parameters + 8.6× data	0.110 (Wikipedia)	\$6
corpus cleaning	0.054	\$8
v4 recipe	0.022	\$9
fine-grained experts	0.008	\$10
3.3× unique data at fixed compute	0.007	\$11
\$12 preset	0 at −24% active FLOPs	\$12
bias_update_rate 1e-3 → 3e-3	0.0015	\$16
prose filtering	−0.023	\$13
sliding window on the lower half	0 at 512, −0.07 at 2048	\$15

Best on Wikipedia: ckpt_hindi_full (0.6586). Best across registers, and the one to build on: ckpt_hindi_mixed — combo preset, 364M / 140M active, 0.7068 / 0.7660.

18. More steps on more data: the mixed corpus at 3× the budget

Same configuration as \$17 — combo preset, Wikipedia + Wikisource, plus \$16's bias_update_rate=3e-3 — at **18,000 steps** (73.7M tokens, 0.52 epochs of the 140.6M) with a fresh cosine schedule, so it is a clean 3×

compute comparison. 1 h 56 min.

	ckpt_hindi_full (Wikipedia, 6k)	ckpt_hindi_mixed (6k)	ckpt_hindi_mixed18k
own val (mixed split)	—	4.9030	4.3017
Wikipedia bench	0.6586	0.7068	0.6150
Wikisource bench	0.8947	0.7660	0.6794
register gap	0.236	0.059	0.064
mean of both	0.777	0.736	0.647

The curve on the mixed val split: 6.14 (1k) → 5.41 (3k) → 4.95 (6k) → 4.70 (9k) → 4.50 (12k) → 4.36 (15k) → 4.30 (18k), still bending down at the end.

Best on both benchmarks, by the widest margins in the log. On Wikipedia it beats the Wikipedia-only best by 0.044 while seeing half as much Wikipedia; on literature it is 0.215 under every Wikipedia-only model. The mean over registers drops 0.09 from §17's best — more than any single lever since the tokenizer. Which is the point: the model was compute-starved on this corpus at 6k steps (0.17 epochs of data it had never seen), so 3× the steps bought nearly 3× the gain that §11 measured when the extra steps were spent re-reading a small corpus. **Compute pays in proportion to how much of the data is new**, and the ledger's "3.3× data at fixed compute: 0.007" was the fixed-compute half of that sentence.

Samples (--seed 7, 200 tokens): मुंबई\n\n is an invented but internally consistent renaming story ("मेडल", "रेड") that drifts into a tourism paragraph; उसने कहा\n\n is 200 tokens of sustained dialogue in the Premchand register — first and second person, rhetorical questions, a बादशाह entering mid-paragraph — with modern orthography this time; the cricket prompt gives a five-sentence career paragraph with fabricated statistics and a clean EOS at 121 tokens. Still not factual; consistently document-shaped and register-appropriate.

Balancing: imbalance ended at 2.17× — §16's floor for 24 experts, now at 3e-3 for 18k steps, unchanged by the longer schedule.

The ledger, once more

lever	bits/byte (mean of both registers unless noted)	
16k BPE instead of bytes	0.162 (Wikipedia)	§7
3× steps on a mostly-unseen mixed corpus	0.089	§18
20× parameters + 8.6× data	0.110 (Wikipedia)	§6
corpus cleaning	0.054 (Wikipedia)	§8
second register in training	0.042	§17
v4 recipe	0.022 (Wikipedia)	§9
fine-grained experts	0.008	§10
3.3× unique data at fixed compute	0.007	§11
§12 preset	0 at −24% active FLOPs	§12
bias_update_rate 1e-3 → 3e-3	0.0015	§16
prose filtering	−0.023	§13
sliding window	0 at 512, −0.07 at 2048	§15

`ckpt_hindi_mixed18k` — 364M total / 140M active, 0.6150 Wikipedia / 0.6794 Wikisource (1.55 / 1.72 bits/char) — is the model to build on, and its curve had not flattened. The next order of magnitude is the same run with more steps and a third register; both are GPU-hours, not code.

19. Accuracy: a golden set

Everything above is compression. bits/byte says how well a model predicts held-out text; it never says whether the model gets anything *right*, and the samples make clear that fluency and correctness have come apart — every date, population and captaincy in §9–18's samples is invented. So, the first accuracy axis:

`make_golden.py` builds **600 Hindi cloze items** from the two benchmark files (no checkpoint trained on either), one per document, and `eval_golden.py` scores them. An item is a prose prefix of up to 120 words, ending on the word before a blank, and the gold answer is the content word that actually follows. Items are filtered to real sentences — the line must have ten or more words, end on a sentence mark, and carry under 10% digits; documents must be at least half sentence-ended lines and not Marathi, which shares the script and turned up in the first draft as a list of plays. Four cells of 150: Wikipedia and Wikisource, each split into

- **in_context** — the gold word already occurred earlier in the prefix, so the passage alone answers it (LAMBADA's construction), and
- **novel** — it did not, so only language knowledge can.

Two scores per checkpoint. **EM**: greedy-decode from the prefix, take the first word, strip punctuation, exact match against the gold word — a synonym or an inflection scores zero. **MC**: four-way multiple choice against three distractors drawn from the same file with similar frequency and length and absent from the prefix, ranked by total log-probability of each choice's tokens (the LAMBADA / HellaSwag protocol); chance is 25%. The whole file is 1.07 MB, committed under `nanosarvam/golden/`, deterministic from `--seed 7`.

	<code>ckpt_hindi_mixed18k</code>	<code>ckpt_hindi_full</code>	<code>ckpt_hindi_v4</code>	<code>ckpt_hindi_350m</code> (bytes)
corpus / steps	Wikipedia + Wikisource, 18k	Wikipedia (full), 6k	Wikipedia (30%), 6k	Wikipedia 107 MB, 6k
all 600, EM / MC	11.2% / 85.8%	6.3% / 76.7%	6.7% / 76.5%	1.5% / 51.8%
Wikipedia	12.3 / 87.0	9.7 / 84.0	9.7 / 85.3	2.7 / 60.7
Wikisource	10.0 / 84.7	3.0 / 69.3	3.7 / 67.7	0.3 / 43.0
in_context	15.7 / 88.3	8.0 / 78.3	8.3 / 77.7	1.7 / 52.3
novel	6.7 / 83.3	4.7 / 75.0	5.0 / 75.3	1.3 / 51.3
Wikipedia / in_context	17.3 / 90.7	12.7 / 86.0	12.0 / 86.0	3.3 / 60.0
Wikipedia / novel	7.3 / 83.3	6.7 / 82.0	7.3 / 84.7	2.0 / 61.3
Wikisource / in_context	14.0 / 86.0	3.3 / 70.7	4.7 / 69.3	0.0 / 44.7
Wikisource / novel	6.0 / 83.3	2.7 / 68.0	2.7 / 66.0	0.7 / 41.3

The byte model sees a quarter of the context (512 bytes $\approx$ 200 characters against 512 tokens $\approx$ 2,000), and its generation budget is 48 bytes so a whole word fits; its first pass at 12 bytes scored 0.0 EM everywhere, a truncation artefact rather than a result, which is why the scorer scales the budget for byte checkpoints.

Six readings.

- **MC is the number to quote.** 86% against a 25% floor: the best model reliably knows which of four plausible Hindi words fits a sentence. EM is low by construction — reproducing the one word the author

wrote, with no credit for synonyms, is hard for a fluent human on the novel items — and is useful for ranking models, not as an absolute.

- **The register result transfers to accuracy.** On literature the mixed model is 15 MC points ahead of the Wikipedia-only model and 3× its EM; on Wikipedia the two are 3 points apart. Same shape as §17's bits/byte.
- **More Wikipedia did not help accuracy.** The 30%-dump and full-dump models are indistinguishable (76.5 vs 76.7 MC, 6.7 vs 6.3 EM) — §11's 0.007 bits/byte at fixed compute, seen from the other side. The whole accuracy gain in the table is the second register and the longer schedule.
- **The tokenizer is worth 34 MC points.** The byte model at the same trunk and budget scores 51.8% — above chance, so it knows some Hindi, but §7's 0.162 bits/byte is here the difference between a coin-flip-and-a-half and reliable: less context per window, and a word is a 15-byte sequence it has to get right byte by byte.
- **Copying is worth less than knowing.** in_context items are easier for both models, but the MC gap between the two types is 5–8 points; most of the 86% is language knowledge, not induction-head copying. On EM the gap is 2× — when the exact word is on the page the model can fetch it.
- **This is still not factual accuracy.** A cloze on held-out text scores whether the model knows Hindi, not whether it knows the world. Question answering with verified answers is the next set to build, and it is a data-writing job; the scorer already takes any JSONL with `prefix`, `answer`, `choices` and `answer_idx`.

Chance for MC was checked the honest way: an untrained tiny model scores it in the test suite (`golden_items_are_well_formed_and_the_scorer_runs`), which also pins the set's contract — the gold word is in the prefix exactly when the item says so, no distractor is, and the blank is the very next word.

20. Fine-tuning to answer questions

The base model continues text; asked a question it produces the article or story that would follow those words. This section teaches it the other thing, with the only instruction data a stdlib-only project can make for itself: **the corpus's own subjects and lead sentences**. `make_qa.py` turns every Wikipedia article into one pair — a question about its title in one of six phrasings, and the article's first one or two sentences as the answer —

प्रश्न: काशी क्या है?
उत्तर: काशी नगरी वर्तमान वाराणसी शहर में स्थित पौराणिक नगरी है।

56,478 pairs from 71,073 articles (the rest have no lead that reads as a definition), mean answer 173 characters, split **by article** 98 / 2 so the 1,129 held-out questions are about subjects whose article the model saw in pretraining but whose question it never did. That is what "answering" can mean at 364M: recall of what it read, in a format it was shown.

`finetune.py` packs the pairs into 6,806 rows of 512 tokens (an example never straddles two rows), masks the prompt to -1 so the **loss lands on answer tokens only** (73% of positions), and otherwise runs `train.py`'s machinery from `ckpt_hindi_mixed18k`: AdamW at lr 5e-5, cosine to 5e-6, 2 epochs = 1,702 steps at batch 8, bf16, grouped dispatch, gradient checkpointing, bias updates after every step. 15 minutes.

held-out answer loss	
before	3.719
step 249	3.325

held-out answer loss	
step 999	3.222
step 1701 (end)	3.202

Most of the drop is in the first 250 steps — the format — and the curve flattens toward the model's ordinary loss on Wikipedia lead sentences.

`eval_qa.py` scores the same held-out questions three ways, with the base model prompted through the same template so the comparison is on one axis. **MC**: the gold answer against three other held-out answers, ranked by mean log-probability per token (chance 25%) — does it know *which* answer belongs to the question. **F1**: SQuAD's token F1 between the greedy answer and the gold one. **subj**: does the decoded answer even name the subject asked about.

	ckpt_hindi_mixed18k (base)	ckpt_hindi_qa
MC (400 questions)	43.0%	50.2%
token F1 (100 decoded)	6.3	21.8
names the subject	12%	81%
answer shape	runs on for 96 tokens, loops, or answers a different question	one to two sentences, encyclopaedic, stops at EOS

Three readings.

- **It learned to answer.** Subject-mention went from 12% to 81% and F1 tripled: the model now produces a definition of the thing asked about, in the register of a lead sentence, and stops. The base model, given the same prompt, mostly wandered into narrative or repeated a phrase 96 tokens long.
- **It did not learn much more than it already knew.** MC moved 43 → 50 against a 25% floor. The base model could already half-recognise which answer belonged to which subject; the fine-tune added seven points, which is format (it now expects an answer to follow) more than recall. Decoded answers name the subject and then attach it to a *plausible category* — an obscure cricketer becomes a television serial, a saint a BJP politician, a state song a Sony show — because at 0.5 epochs a 364M model has the shape of Wikipedia but few of its facts. Every date and place in the answers should be read as fiction.
- **Greedy decoding loops.** Two of six printed samples degenerate into a repeated phrase. `generate()` now takes `repetition_penalty` (CTRL's rule, off by default; the tests confirm 1.0 is bit-identical to before), and `serve.py`'s question mode uses 1.3, which breaks the loops without changing the answers that were already fine.

What would move MC: more pretraining (the corpus was seen half a time), and question types that are not definitions. The pipeline is in place for both — `make_qa.py` is one template away from "where is X" or "when was X born", and `eval_qa.py` scores any file with `question / answer / title`.

`serve.py --ckpt ckpt_hindi_qa.pt` serves the result: the page detects the checkpoint's `qa_template` and offers *answer a question* beside *continue the text*, the base behaviour, on the same weights.

21. Twice the steps: 18k → 36k, and what reaches the answers

§18 ended with "the curve had not flattened" and §20 with "what would move MC is more pretraining". So: the same run, continued. `ckpt_hindi_mixed18k` was copied to a new name and `train.py --resume --steps 36000` picked it up at step 18,000 — weights, AdamW moments and sampler position — with the cosine now spanning 36k steps. Extending a *finished* run lands mid-cosine, at $3e-4$ against the $3e-5$ the weights had annealed to, so `--rewarmup 500` (new) ramps the lr from zero back to the schedule over 500 steps instead of jumping. 18,000 more steps, 73.7M tokens, 1 h 58 min; the model ends at epoch 1.17, so its second half was still 80% first-pass text.

The restart costs a bump and pays it back. Held-out loss on the mixed split, same windows as every earlier eval:

step	18k (start)	19k	21k	23k	25k	26k	29k	32k	36k
val	4.302	4.507	4.456	4.385	4.316	4.287	4.197	4.141	4.099

The first eval is 0.2 *worse* than the checkpoint it resumed from: the lr ramp undid some of the anneal, as a warm restart does. The right comparison is against the original run at matched lr — 4.70 at step 9k, lr $2.5e-4$ — and by that measure the resumed run was ahead throughout. It crossed below the old best at step 26k and set a new one at every eval after.

Benchmarks

	<code>ckpt_hindi_mixed18k</code>	<code>ckpt_hindi_mixed36k</code>	Δ
Wikipedia bits/byte	0.6150	0.5876	−0.027
Wikisource bits/byte	0.6794	0.6479	−0.032
bits/char	1.548 / 1.718	1.479 / 1.639	
golden set EM / MC (§19)	11.2 / 85.8	12.7 / 86.3	+1.5 / +0.5
golden, Wikipedia items	12.3 / 87.0	15.3 / 88.7	
golden, Wikisource items	10.0 / 84.7	10.0 / 84.0	

−0.03 bits/byte on both registers — more than the recipe change (§9), fine-grained experts (§10) and the data lever (§11) together, and the second-largest single move in the log after the tokenizer. The golden set moved a point: its MC column is near saturation at 86%, where the items left are ones with several genuinely plausible choices, and its stricter EM column moved 3 points on Wikipedia and none on literature. Compression keeps improving after the cloze test stops being able to see it.

The same fine-tune, from the better base

§20's recipe unchanged — the same 55,349 pairs, 2 epochs, lr $5e-5$ — from `ckpt_hindi_mixed36k`, giving `ckpt_hindi_qa36k`. 21 minutes.

	base 18k	tuned 18k	base 36k	tuned 36k
held-out answer loss	3.719	3.202	3.488	2.984
MC, answer recognition (chance 25%)	43.0	50.2	43.5	51.2
token F1	6.3	21.8	8.4	23.0

	base 18k	tuned 18k	base 36k	tuned 36k
names the subject	12	81	36	86

Three readings.

- **Pretraining carries straight through.** The tuned 36k model's answer loss is 0.22 below the tuned 18k model's — the entire gap between the two bases, preserved by the fine-tune. And the untuned 36k base already names the subject three times as often as the 18k base did (36% vs 12%) when handed the question template: the extra pretraining alone made it more willing to talk about the thing it was asked about.
- **Recognition barely moves.** MC went 50.2 → 51.2. Twice the pretraining compute, −0.03 bits/byte, and the model is one point better at telling which of four lead sentences belongs to a subject. F1 +1.2, subject mention +5. What improves with compute is *how* it answers; *what* it knows about any particular subject is set by how many times that subject appeared in 140M tokens, which is usually once. The decoded answers are the same kind of wrong as §20's: fluent, the right shape, a plausible category, invented dates — and one repeated-name loop in six samples even now.
- **This is the ceiling of the corpus, not the model.** A subject that appears in one article, read 1.2 times, is not a memory; it is a perplexity. Moving MC materially means more mentions per fact — a larger corpus with redundancy, or many epochs — or retrieval, where the lead sentence is looked up rather than recalled. Both are further than this log goes.

The ledger, closed once more

lever	bits/byte (mean of both registers unless noted)	
16k BPE instead of bytes	0.162 (Wikipedia)	\$7
3× steps on a mostly-unseen mixed corpus	0.089	\$18
20× parameters + 8.6× data	0.110 (Wikipedia)	\$6
corpus cleaning	0.054 (Wikipedia)	\$8
second register in training	0.042	\$17
2× steps again, 18k → 36k, warm-restarted	0.030	\$21
v4 recipe	0.022 (Wikipedia)	\$9
fine-grained experts	0.008	\$10
3.3× unique data at fixed compute	0.007	\$11
§12 preset	0 at −24% active FLOPs	\$12
bias_update_rate 1e-3 → 3e-3	0.0015	\$16
prose filtering	−0.023	\$13
sliding window	0 at 512, −0.07 at 2048	\$15

ckpt_hindi_mixed36k — 364M / 140M active, 0.5876 / 0.6479 bits/byte, golden 12.7 / 86.3 — is the model to build on, and **ckpt_hindi_qa36k** the one to ask questions of; `serve.py` runs it. Its curve was still bending down at 36k (4.114 → 4.106 → 4.099 over the last three evals), more gently than at 18k. The step after this one is a bigger corpus, not a longer schedule.